Java

Page No:
Date. 01 07 2026

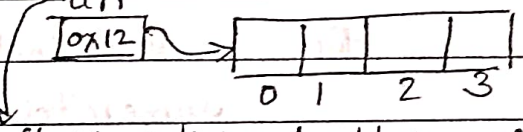
* **Variable** → • a memory block that is used to store data.
  • variable can store only one value

declaration
initialization

* **Data Types**

Primitive
- stores actual data
- int age = 21;

Non-Primitive
- Stores reference or address
- arr



stores reference / address, not the actual value

Primitive Data Types

		Bytes	Default
Integer	byte	1	0
	short	2	0
	int	4	0
	long	8	0L
	float	4	0.0f
	double	8	0.0 / 0.0d
	char	2	'\u0000' (unique escape sequence)
	boolean	1 bit	false
Any Reference (eg. String, object, arrays)			null

float f1 = 2.5 f

* **Operators**

Increment (++)

- Post Increment (operand++)

```
int a = 25;
int b = a++;
```

a: 25 → 26

a = 26
b = 25

b: 25

(Use value first & then update)

- Pre Increment (++operand)

```
int b = -10;
int c = ++b;
```

b: -10 → -9

b = -9
c = -9

c: -9

(First update & then use value)

Decrement (- -)

- operand -- (Post Decrement)
- -- operand (Pre Decrement)

Page No:
Date:

* Finding last digit

`num %10` Eg: 123 %. 10 = 3

* 2 %. 5 = 2

when the numerator is smaller than the denominator, the result will be numerator

* Conditional Statements / Decision statements

if, if else, Else if ladder, nested if, switch

if → only for positive condition
else → ~~tobe~~ for negative condition

* Compilation → starts from first line
execution → starts from main method

compilation → javac filename.java
execution → java Classname

JVM use .Class file to execute
↓ contains bytecode

* Loops

while, do while, for, foreach

- while (condition) {
  - - -
  - - -
  updation stmt;
  }

Eg:
```
int i=1;
while (i<=4){
    S.O.P("Hello");
    i++;
}
```

Page No:
Date.

tracing → "Use Tabular format"

i	i<=10	O/P	i++
1	T	Hello	2
2	T	Hello	3
3	T	Hello	4
4	T	Hello	5
5	F	———	———

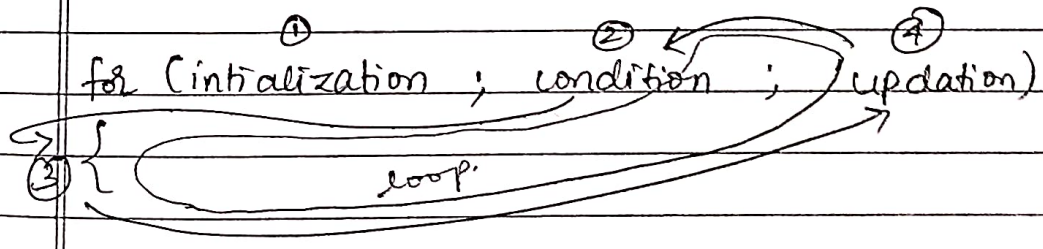
- do {
  ...
  ...
  } while (condition);

do while
- Minimum no. of execution statement is '1'

while
- Minimum no. of execution statement is '0'

Eg:
```
int i=1;
do {
    S.O.P("Hello");
    i++;
} while (i<=5);
```

- for loop



```
for( ; ; )
{
}
```

Because by default the Condition is ~~false~~ true in for loop

Page No:
Date.

On while & do-while condition is ~~mandatory~~.

Programs

Page No:
Date.

1) Swap Two numbers using Temporary variable

```
public class SwapTemp{
    Public static void main(String[] args){
        int a =10;
        int b = 20;

        int temp=a;
        a = b;
        b = temp;
        S.O.P ("After swapping :");
        S.O.P ("a :"+a);
        S.O.P ("b :"+b);
    }
}
```

2) Swap Two numbers without using temp

(i) Using Addition and subtraction

```
a = a+b;
b = a-b;
a = a-b;
```

a = 10	b = 20
a = 30	(10 + 20)
b = 10	(30 - 20)
a = 20	(30 - 10)

(ii) Using multiplication and subtraction

```
a = a *b;
b = a /b;
a = a/b;
```

a = 10 b = 20

a = 10 * 20 = 200

b = 200/20 = 10

a = 200/10 = 20

(iii) Using XOR

```
a = a^b;
b = a^b;
a = a^b;
```

a = 10, b = 20

10 = 1010

20 = 10100

a = a^b

= 10^20

```
  1 0 1 0
  1 0 1 0 0
  ---------
  1 1 1 1 0  = 30
```

b = a^b

= 30^20

```
  1 1 1 1 0
  1 0 1 0 0
  ---------
  0 1 0 1 0  = 10
```

a = a^b

a = 30^10

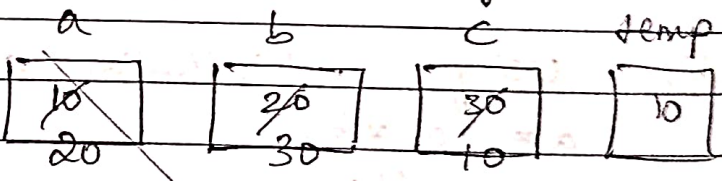
```
  1 1 1 1 0
  0 1 0 1 0
  ---------
  1 0 1 0 0  = 20
```

∴ a = 20, b = 10

Assign8

a) Swap three numbers using Temporary variable

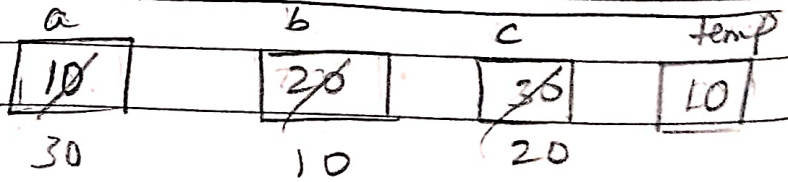
```
int temp = a;
a = b;
b = c;
c = temp;
```



a = 10, b = 20, c = 30

a = 20, b = 30, c = 10

```
int temp = a;
a = c;
c = b;
b = temp;
```

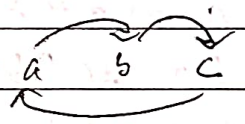


```
public class SwapThree{
public static void main(String[] args){
    int a = 10;
    int b = 20;
    int c = 30;

    int temp = a;          int temp = a;
    a = b;                 a = c;
    b = c;                 c = b;
    c = temp;              b = temp;
    S.O.P("After swapping :");
    S.O.P("a = "+a);
    S.O.P("b = "+b);
    S.O.P("c = "+c);
  }
}
```

b) Swap three numbers without using a temporary variable

(i)
```
a = a+b+c;
b = a-(b+c);
c = a-(b+c);
a = a-(b+c);
```



a = 10, b = 20, c = 30

a =	10 + 20 + 30 = 60	[a = 60, b = 20, c = 30]
b =	60 - (20 + 30) = 10	[a = 60, b = 10, c = 30]
c =	60 - (10 + 30) = 20	[a = 60, b = 10, c = 20]
a =	60 - (10 + 20) = 30	[a = 30, b = 10, c = 20]

(ii) a = a ^ b ^ c
b = a ^ b ^ c
c = a ^ b ^ c
a = a ^ b ^ c

a = 10, b = 20, c = 30

a = a ^ b ^ c => (10 ^ 20) ^ 30
=> 30 ^ 30 = 0

b = a ^ b ^ c => 0 ^ 20 ^ 30
=> 0 ^ 10
=> 10

c = a ^ b ^ c => 0 ^ 10 ^ 30
=> 0 ^ 20
=> 20

a = a ^ b ^ c => 0 ^ 10 ^ 20
=> 0 ^ 30 = 30

a = 30
b = 10
c = 20

(iii) a = a * b * c
b = a / (b * c)
c = a / (b * c)
a = a / (b * c)

a = 2 b = 3 c = 4
a = 2 x 3 x 4 = 24
b = 24 / (3 x 4) = 24 / 12 = 2
c = 24 / (2 x 4) = 24 / 8 = 3
a = 24 / (2 x 3) = 24 / 6 = 4

39) Program to print the first 100 natural numbers

Page No:
Date.

```
public class NaturalNumbers {
    public static void main(String[] args) {
        for(int i=1; i<=100; i++) {
            S.O.P(i);
        }
    }
}
```

- Taking 'n' value from user

```
Scanner sc = new Scanner(System.in);
S.O.P("Enter n value:");
int n = sc.nextInt();
```

40) Program to print the sum of first 'n' natural numbers

```
import java.util.Scanner;

public class SumNaturalNumbers {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        S.O.P("Enter n value:");
        int n = sc.nextInt();

        int sum=0;
        for(int i=1; i<=n; i++) {
            sum += i;
        }
        S.O.P("Sum = " + sum);
    }
}
```

Using Formula

```
int n = sc.nextInt();
int sum = n*(n+1)/2;
S.O.P ("Sum = " + sum);
```

5) Program to print all the digits of a given number

```
public class PrintDigits {
    public static void main (String[] args){
        int num = 1235;
        while (num!=0) {
            int lastDigit = num%10;
            S.O.P (lastDigit);
            num = num/10;
        }
    }
}
```

I/P 1 2 3 5 O/P 5 3 2 1

lastDigit = 5	lastDigit = 3	lastDigit = 2	lastDigit = 1
num = 123	num = 12	num = 1	num = 0

6) Program to find the sum of digits of a number

```
int num = 1235;
int sum = 0;
while (num!=0) {
    int lastDigit = num%10;
    sum += lastDigit;
    num = num/10;
}
S.O.P (sum);
```

7) Program to find the products of digits of a given number

Page No:
Date:

```
int num = 1235;
int prod = 1;
while (num != 0) {
    int rem = num % 10;
    prod *= rem;
    num = num / 10;
}
S.O.P (prod);
```

8) Program to reverse a number

123
321

```
int num = 123;      int rev = 0;
while (num != 0){
    int lastdigit = num % 10;
    rev = rev * 10 + lastdigit;
    num = num/10;
}
S.O.P (rev)
```

1 * 3

3 0 +

2 * 100

200 +

9) Program to check whether the given number is palindrome or not

Page No:
Date.

```
int num = 121;
int rev = 0;
int originalNum = num;
while (num > 0) {
    int lastdigit = num % 10;
    rev = rev * 10 + lastdigit;
    num = num / 10;
}
if (originalNum == rev) {
    S.O.P ("Palindrome");
}
else {
    S.O.P ("Not Palindrome");
}
```

Palindrome

Given a number, if the reverse of the given number is exactly equal to the original Number, then those numbers are called as Palindrome numbers.

Assgn8 Program to check whether the given year is leap year or not.

* Year.isLeap(year) → import java.time.year;
  (2026)
  or
* A year is leap year if it is divisible by 4 but not by 100, OR if it is divisible by 400

```
public class LeapYear {
    public static boolean isLeap (int year) {
        return (year % 400 == 0) || (year % 4 == 0 &&
                year % 100 != 0);
    }
}
```

10) Program to check whether the given number is Sunny Number.

Page No:
Date.

10): Factorial of a given number

5!
5 x 4!
n * fact (n-1)

~~int num = 5~~
~~int fact = 1;~~

~~while (num < 0) {~~

~~fact = fact *~~

```
int num = 5
int fact = 1;

if (num == 0) {
    S.O.P (fact);
}
else {
    for (int i = num ; i > 0; i--){
        fact = fact * i;
    }
    S.O.P (fact);
}
```

11) Program to find factorial of all numbers between m and n.

```
for (int i = m ; i <= n; i++){
    int fact = 1;
    for (int j = i ; j > 0; j--){
        fact = fact * j;
    }
    S.O.P (i + " -> " + fact);
}
```

2 m → 1
n → 10
1 → 1
2 → 2
3 → 6
4 → 24
5 → 120
6 →
7 →
8 →
9 →
10 →

(12) Program to check whether the given number is Strong number or not.

Page No:
Date.

Strong Number

Given a number, if the sum of the factorial of each digit in the given number is exactly equal to the original number, then those numbers are called as Strong Numbers.

145

5! → 120

4! → 24

1! → 1

145

```
int num = 145
int originalNum = num;
int sum = 0;
while( num != 0) {
    int lastdigit = num % 10;
    int fact = 1;
    for (int i = lastdigit ; i >= 0 ; i--){
        fact = fact * i;
    }
    sum = sum + fact
    num = num/10;
}
if ( sum == temp)
{
    S.O.P ("Strong Number")
}
else {
}
```

(a) Strong numbers b/w the range m and n

(b) Count of Strong numbers b/w the range m and n

Page No:
Date:

(13) Program to print the factors of given number.

6 → 1, 2, 3, 6

5 → 1, 5

6	6 % 6 = 0
5	6 % 5 = 1
4	6 % 4 = 2
3	6 % 3 = 0
2	6 % 2 = 0
1	6 % 1 = 0

```
int num = 6;
for (int i = num; i > 0; i--) {
    if (num % i == 0) {
        S.O.P (i);
    }
}
```

- b/w range

```
int m = 1;
int n = 5;
for (num = m; num <= n; num++){
    for (int i = 1; i <= num; i++){
        if (num % i == 0) {
            S.O.P (i + " ")
        }
    }
    S.O.P();
}
```

(14) Program to print count of factors of a given numbes.

```
int num = 6; int count = 0;
for (int i = num; i > 0 ; i--){
    if (num % i == 0){
        count++;
    }
}
S.O.P (count);
```

(15) Program to check whether the given No. is prime & not.

```
int num = 2;
int count = 0;
for (int i = num; i > 0; i--)
{
    if (num % i == 0)
    {
        count++;
    }
}
if (count == 2) {
    S.O.P ("Prime");
}
else {
    S.O.P ("Composite");
}
```

(16) Prime Numbers b/w the range m and n

```
int m = 2;
int n = 5;
for (int i = m; i <= n; i++) {
    for (int j = i; j > 0; j--) {
        if (num % j == 0) {
            count++;
        }
    }
    if (count == 2){
        S.O.P(i);
    }
}
```

(17) Program to print first 15 Prime numbers.

```
int count = 0;
int num = 1;
while (count <= 15){
int c = 0;
for (int i = num; i > 0; i--){
    if (num % i == 0){
        c++;
    }
}
if (c == 2){
    S.O.P (num);
    count++;
}
num++;
}
```

1
2
3
4
5

(18) Program to print first 25 two digit prime no.

```
int count = 0;
int num = 10;
while {

}
```

(19) Program to print last 25 two digit prime no

```
int num = 99;

num--;
```

(20) Write a program to print the first 15 numbers of fibonacci series.

Page No:
Date.

0 1 1 2 3 5 8

f(n-1) + f(n-2)

3 + 2

= 5

0 1 1 2 3 5 8

f(n-1) + f(n-2)

4 + 3

```
int n1 = 0;
int n2 = 1;
S.O.P(n1);
S.O.P(n2);
int count = 2;
while (count <= 20)
{
  int n3 = n1 + n2;
  S.O.P(n3);
  n1 = n2;
  n2 = n3;
  count++;
}
```

(21) Program to check whether the given number is perfect square or not.

~~int num = 25;~~

= ~~Math.sqrt~~

4

1 * 1 = 1

2 * 2 = 4

3 * 3 = 9

4 * 4 = 16

~~traverse~~ ~~till~~ half

9

1 * 1 = 1

2 * 2 = 4

3 * 3 = 9

~~4 * 4 = 16~~

~~5 * 5 = 25~~

~~6 * 6 =~~

~~7 * 7 =~~

~~8 * 8 =~~

~~9 * 9 =~~

9/2 = 4

```
int num=9;
boolean res=false;
for(int i=1; i<num/2; i++){
    if(i*i==num){
        res=true; break;
    }
}
if(res){
    S.O.P("Perfect Square");
}
}
```

for (int num=m; num<=n; num++){ — for b/w range m & n

Assign b/w range.

(22) Program to check whether the given number Dunny Number.

(8) → Dunny no.

8+1 → (9) perfect square

```
int num=8;
int num1=num+1;
for(int i=1; i<num1/2; i++){
    if(i*i==num1){
        res=true; break
    }
}
if(res){
    S.O.P(num + "is a Dunny number")
}
```

(23) Program to check whether the given number is Neon Number.

Page No:
Date.

Neon Number

81 → 1+8 → 9² → 81

```
int num = 81;   int originalNum = num;   int sum = 0
while (num != 0) {
    int lastdigit = num % 10;
    sum += lastdigit;
    num = num / 10;
}
int SquareNum = sum * sum;
if (SquareNum == originalNum) {
    S.o.P("Neon")
}
```

(24) Armstrong Number

153

count → 3

3^3 → 27

5^3 → 125

1^3 → 1

153

```
int num = 153;
int temp = num;
int temp1 = num;
int count = 0;
int sum = 0
while (temp1 != 0)
{
    count++;
    temp1 /= 10;
}
while (temp != 0) {
    int rem = temp % 10;
    int prod = 1;
    for (int i = 1; i <= count; i++) {
        prod = prod * rem;
    }
    sum = sum + prod;
    temp /= 10
}
```

```
if (sum == num) {
    S.o.p ("Armstrong no");
}
else {
    S.o.p ("not");
}
```

Page No:
Date:

(25) Program to check whether the given no. is Tech numbers or not.

2025

* even no. of digits

2025

20 + 25

$(45)^2 \rightarrow 2025$ //

```
int num = 2025;
int temp = num;
int count = 0;
while (temp != 0){
    count++;
    temp /= 10;
}
if (count % 2 == 0){
    int d = 1;
    for (int i = 1; i <= count/2; i++) {
        d = d * 10;
    }
int firsthalf = num/d;
int secondhalf = num % d;
int sum = firsthalf + secondhalf;
int sq = sum * sum;
if (sq == num) {
}
else {
}
```

neon
Prime
Fib

(26) Automorphic Number

Page No:
Date.

(5) → 5^2 → 25 same as 5

(6) → 6^2 → 36

(25) → 25^2 → 625

```
int num = 25
int originalNum = num;
int count = 0;

while (num != 0){
    count++;
    num /= 10;
}

int squarednum = originalnum * originalnum

int d = 1;
for (int i=0; i < count; i++){
        d = d*10;
}

int final = squarednum % d

if (final == originalNum) {
    s.o.p(" An
}
```

(27) Xylem & Phloem Number

(1) (2) (3) (4)

Page No:
Date.

first + last = 1+4 → (5) → same " " of " "
between = 2+3 → (5)

(28) Duck Number — The number should contain minimum one zero.

1205 → ✓
1340 → ✓
0123 → ✗

(29) Happy Number

19 → $1^2 + 9^2$
$1 + 81$
↓
82

82 → $8^2 + 2^2$
$64 + 4$
68

68 → $6^2 + 8^2$
$36 + 64$
100

100 → $1^2 + 0^2 + 0^2$ = (1) ∴ happy.

(30) Bouncy Number

1 6 4
↓ ↘
increase decrease

1678 → increase
321 → decrease

1↑6↓3↑7↓2 → bouncy.

(27) Xylem and Phoelem

1 2 3 4

1 + 4 = 5
2 + 3 = 5 → xylem

Page No:
Date.

```
import java.util.Scanner;

public class XylemPhloem{
    public static void main (String[] args){
        Scanner sc = new Scanner (System.in);

        S.O.P ("Enter a number :");
        int num = sc.nextInt();

        int extremeSum = 0;
        int meanSum = 0;

        int lastDigit = num %10;
        num = num/10;

        while (num >=10) {
            int digit = num %10;
            meanSum += digit;
            num = num/10;
        }

        int firstDigit = num;

        extremeSum = firstDigit + lastDigit

        if (extremeSum == meanSum) {
            S.O.P ("Xylem Number");
        } else {
            S.O.P ("Phoelem Number");
        }
    }
}
```

(28) Duck Number

1205 ✓
1340 ✓
0321 ✗

Page No:
Date.

↙

* A number starting with 0 is interpreted as an octal (base-8 literal)

* An int cannot preserve leading zeros.

* If leading zeros matter, we can use String.

```
Scanner sc = new Scanner (System.in);
S.O.P ("Enter a number: ");
String num = sc.next();

if (num.charAt(0) == '0') {
    S.O.P ("Not Duck number");
    return;
}
if (num.contains ("0")) {
        S.O.P ("Duck Number");
}
else {
    S.O.P ("Not Duck Number");
}
```

```
int num = 5021;
boolean isDuck = false;
while (num > 0) {
    if (num % 10 == 0) {
        isDuck = true;
        break;
    }
    num = num / 10;
}
if (isDuck) {
    S.O.P ("Duck Number")
}else{ S.O.P ("Not a Duck Number")
```

(29) Bouncy Number

Page No:
Date.

```
Scanner sc = new Scanner(System.in);
S.O.P("Enter a number:");
int num = sc.nextInt();

boolean increasing = false;
boolean decreasing = false;

int lastDigit = num%10
num /=10;

while (num>0){
    int currentDigit = num%10

    if (currentDigit < lastDigit)
        increasing = true;

    if (currentDigit > lastDigit)
        decreasing = true;

    lastDigit = currentDigit;
    num /=10;
}

if (increasing && decreasing){
        S.O.P("Bouncy Number");
}else{
    S.O.P("NOT Bouncy Number");
}
```

(30) Happy Number

```
Scanner sc = new Scanner(System.in);
S.o.p("Enter number: ");
int num = sc.nextInt();

while (num != 1 && num != 4){

    int sum = 0

    while (num > 0) {
    int digit = num % 10
        sum = sum + digit * digit;
        num /= 10
    }
    num = sum;
}

if (num == 1){
    S.o.p("Happy Number");}
else {
    S.o.p("Not Happy Number");
}
```

19 → $1^2 + 9^2$
$1 + 81$
82

82 → $8^2 + 2^2$
$64 + 4$
68

68 → $6^2 + 8^2$
$36 + 64$
100

100 → $1^2 + 0^2 + 0^2 = 1$ (Happy)

(20)
$2^2 + 0^2 = 4$
$4^2 = 16$
$1^2 + 6^2 = 37$
$3^2 + 7^2 = 58$
$5^2 + 8^2 = 89$
$8^2 + 9^2 = 145$
$1^2 + 4^2 + 5^2 = 42$
$4^2 + 2^2 = 20$
again repeats
$2^2 + 0^2 = 4$

Every unhappy number enters this cycle, & 4 is always part of it

12a) Strong numbers b/w the range m and n

12b) count of strong no. b/w m & n.

Page No:
Date.

```
int m = 1
int n = 50
int count = 0;
for (int num = m; num <= n; num++){

    int temp = num;
    int sum = 0;

    while (temp > 0) {
        int lastDigit = temp % 10;
        int fact = 1;
        for (int i = 1; i <= lastDigit; i++){
            fact = fact * i;
        }
        sum = sum + fact;
        temp = temp / 10;
    }
    if (sum == num) {
        S.O.P (num);    count++
    }
}
S.O.P (count);
```